

MakeCode Graphical Programming

MakeCode Graphical Programming

1. Introduction to MakeCode Platform
2. Getting Started with MakeCode
3. Importing Lastbit Exclusive Extension Package
 - 3.1 Import Using Existing Project (Recommended)
 - 3.2 Import in Empty Project
4. Flashing Programs
 - 4.1 Flashing Example Programs
 - 4.2 Flashing Custom Programs
5. Experimental Phenomena
6. Precautions

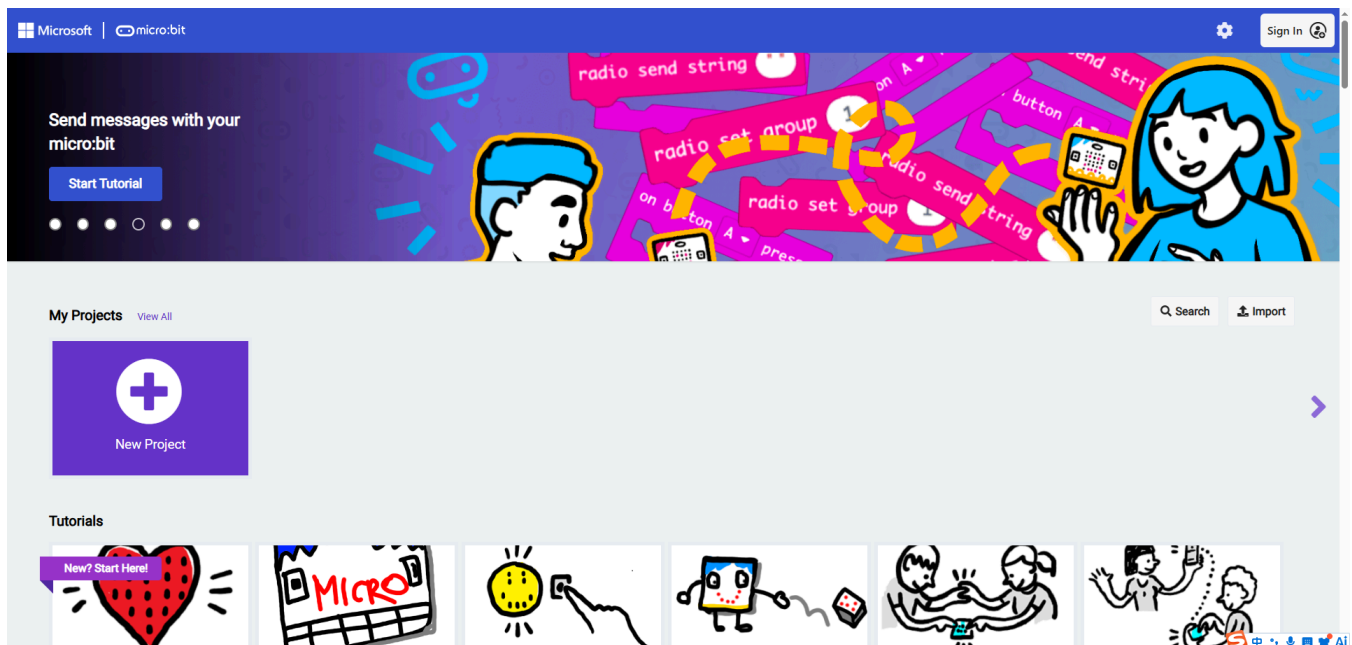
1. Introduction to MakeCode Platform

Microsoft MakeCode editor is the mainstream tool for BBC micro:bit programming and creation. The editor distinguishes functional modules through different colors and provides access to all micro:bit features. In addition to the graphical programming interface, MakeCode also supports switching to JavaScript to view the text code corresponding to the current block program.

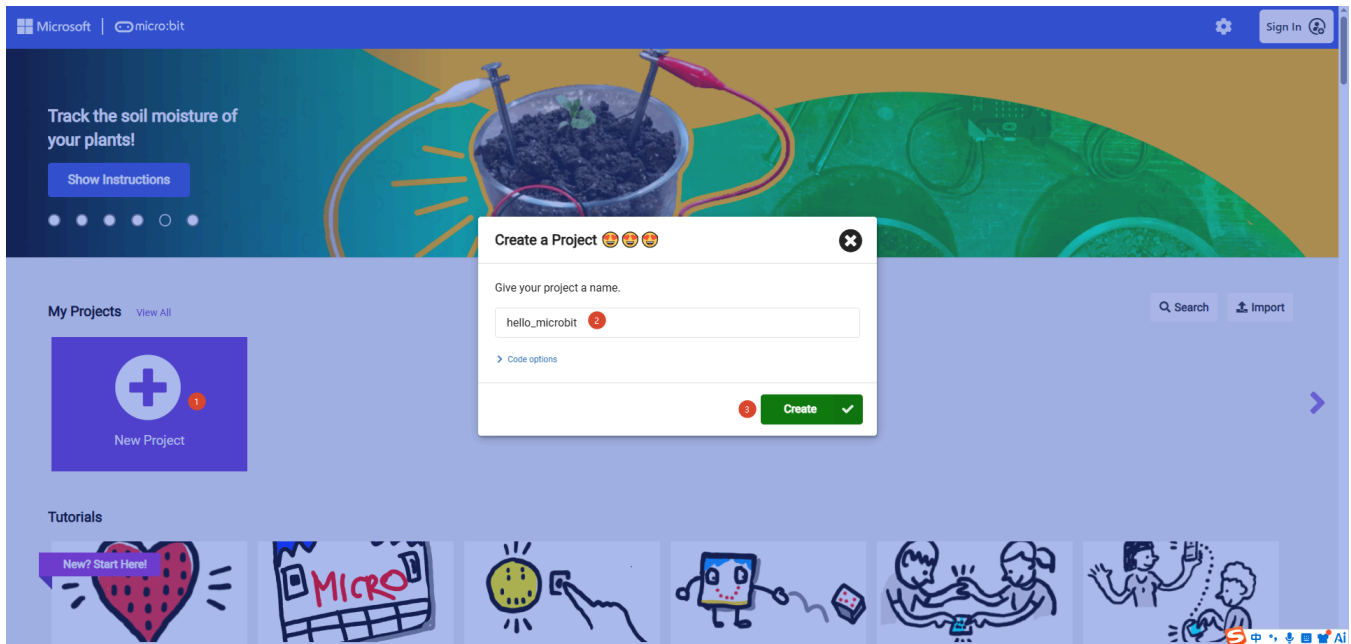
2. Getting Started with MakeCode

1. Open the link below to access the MakeCode editor:

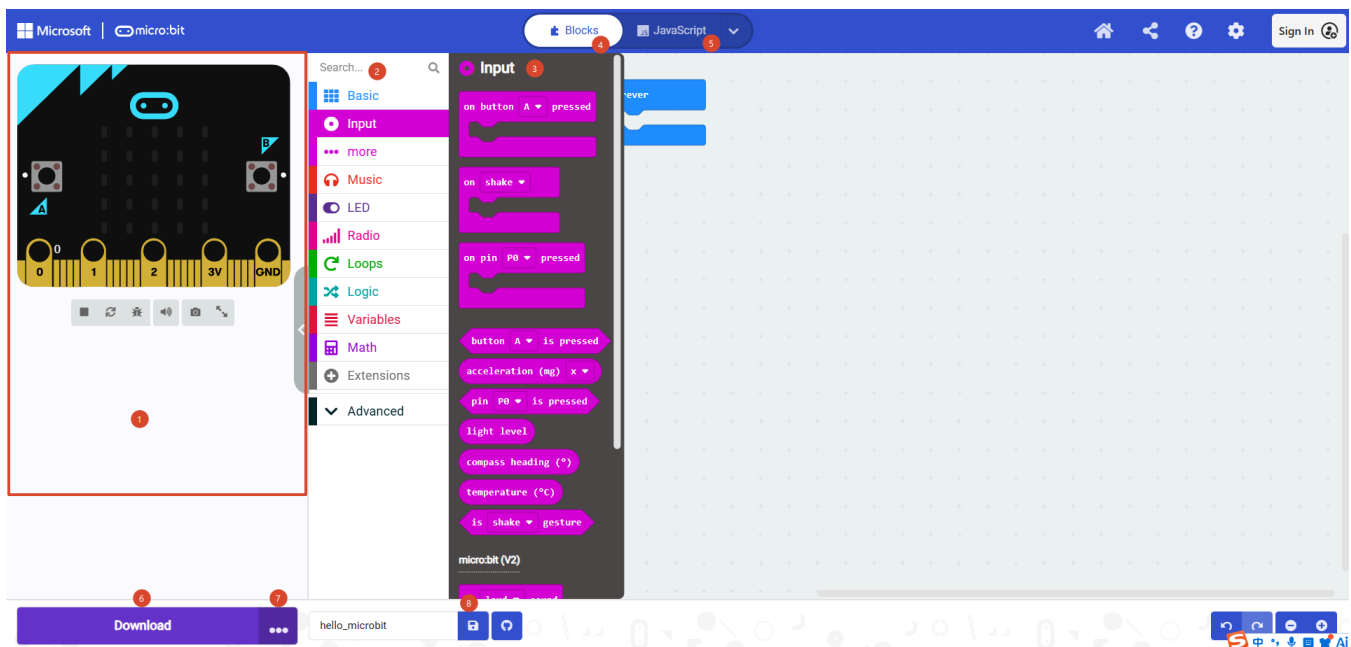
[Microsoft MakeCode for micro:bit](#)



2. Click "New Project", enter a project name (e.g., `hello_microbit`), then click "Create" to enter the programming environment.



3. The MakeCode editing interface consists of the following areas:



- ① **Simulation Area:** Used to preview program execution effects. The A, B buttons and logo button (micro:bit V2 only) shown in the figure support clicking.
- ② **Block Category Area:** Displays block categories by function. The Lastbit car provides an exclusive extension package; after importing, you can view related blocks in this area.
- ③ **Block Area:** Displays all blocks under the current category. You can drag the required blocks into the programming area.
- ④ **Programming Area:** Used to combine blocks and build program logic.
- ⑤ **Code Switch Area:** Supports switching to JavaScript to view the text code corresponding to graphical blocks.
- ⑥ **Download Area:** Used to connect Micro:bit to computer and download programs.
- ⑦ **More:** You can choose to connect to Micro:bit, or save the program as `microbit-project_name.hex`, for example `microbit-hello_microbit.hex`.

⑧ **Save Button:** Saves the program as `microbit-project_name.hex`, e.g., `microbit-hello_microbit.hex`.

3. Importing Lastbit Exclusive Extension Package

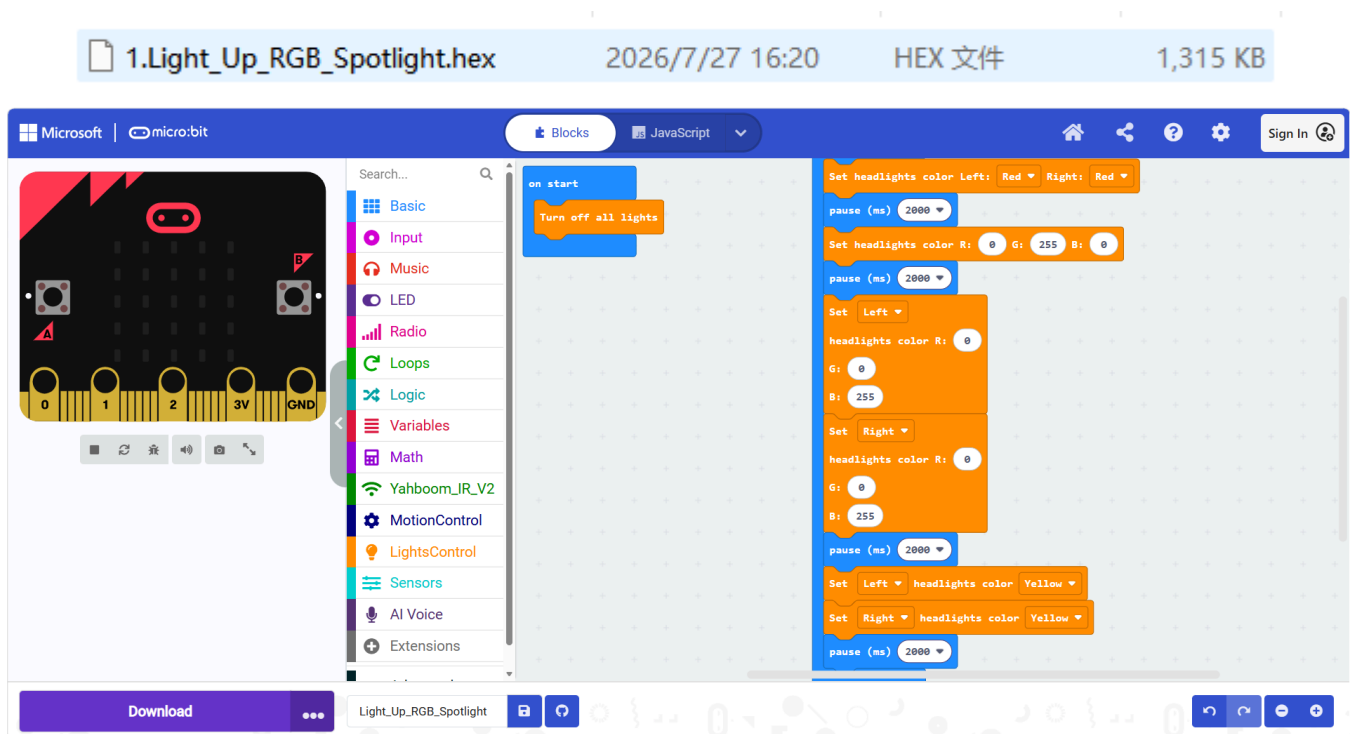
The Lastbit car provides an exclusive extension package. Before programming, please import the extension package into your project first. Below are two import methods.

Note

The AI voice interaction integration module extension package and corresponding examples require optional selection to use

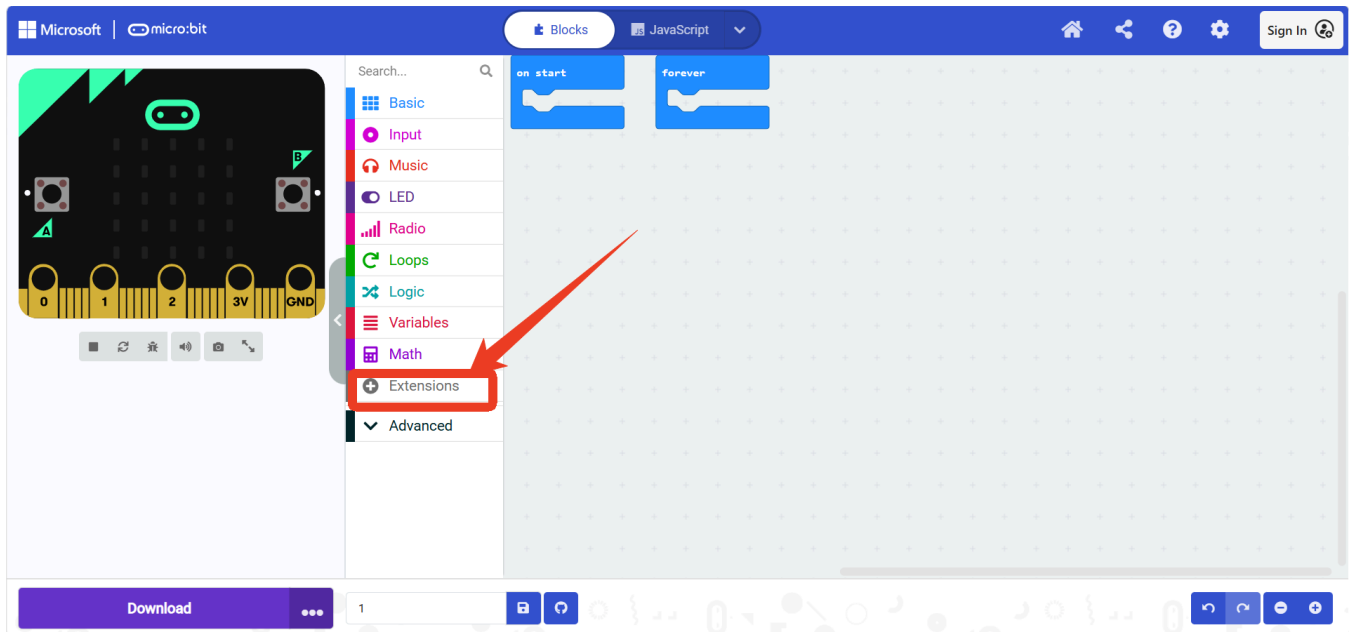
3.1 Import Using Existing Project (Recommended)

1. Find the `.hex` file we provided. Using 1.Light_Up_RGB_Spotlight as an example.
2. Drag the file to the MakeCode page, and the project will be automatically imported and opened.

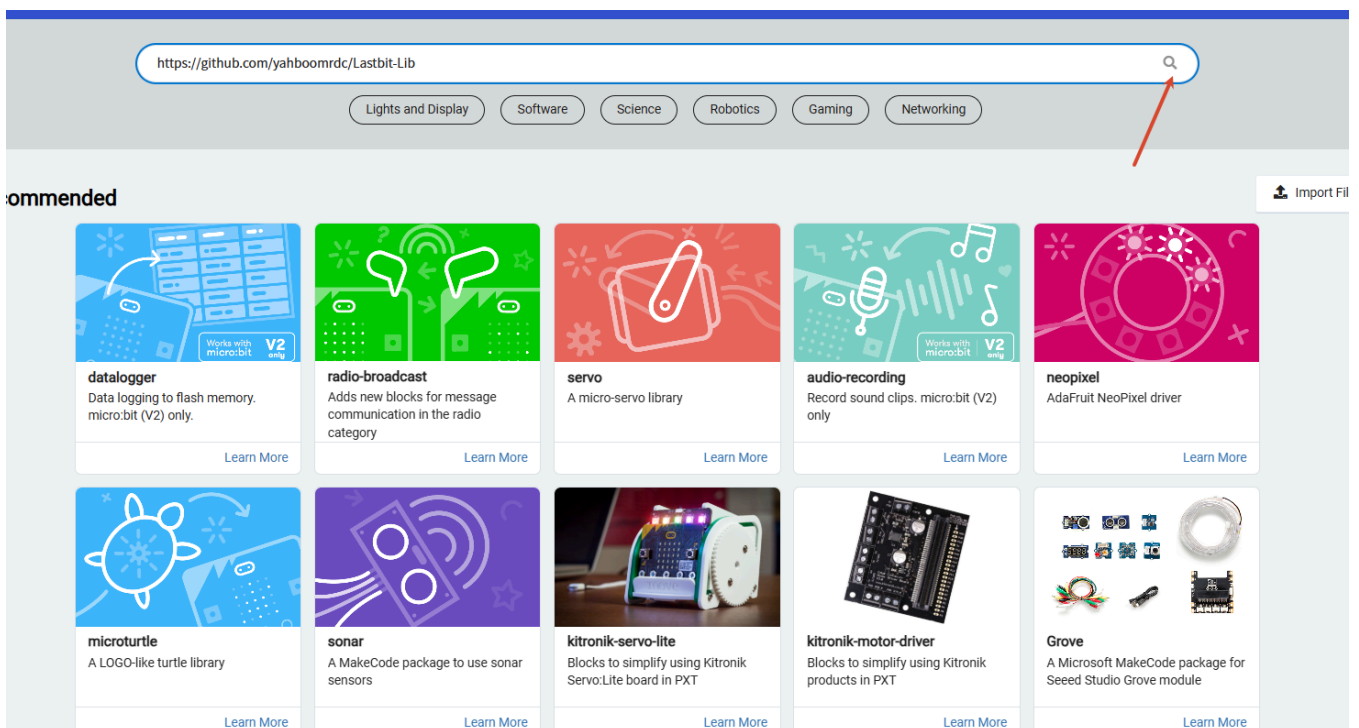


3.2 Import in Empty Project

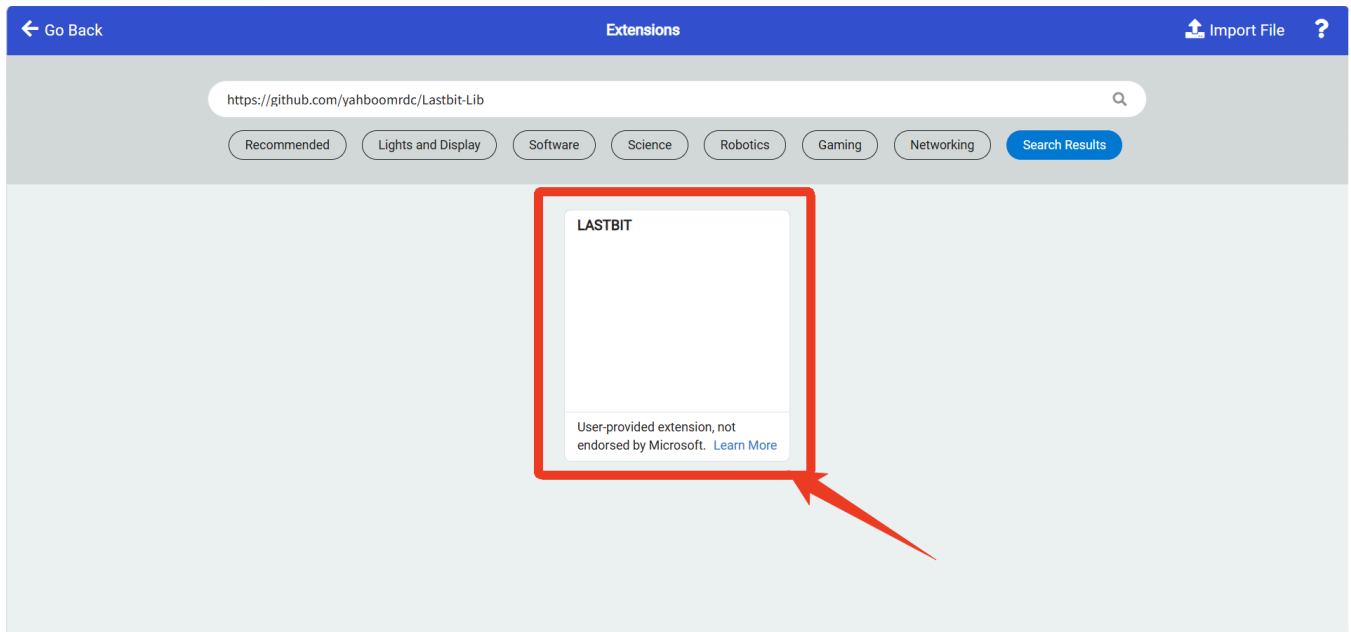
1. Click the "Extensions" option at the bottom of the category bar.



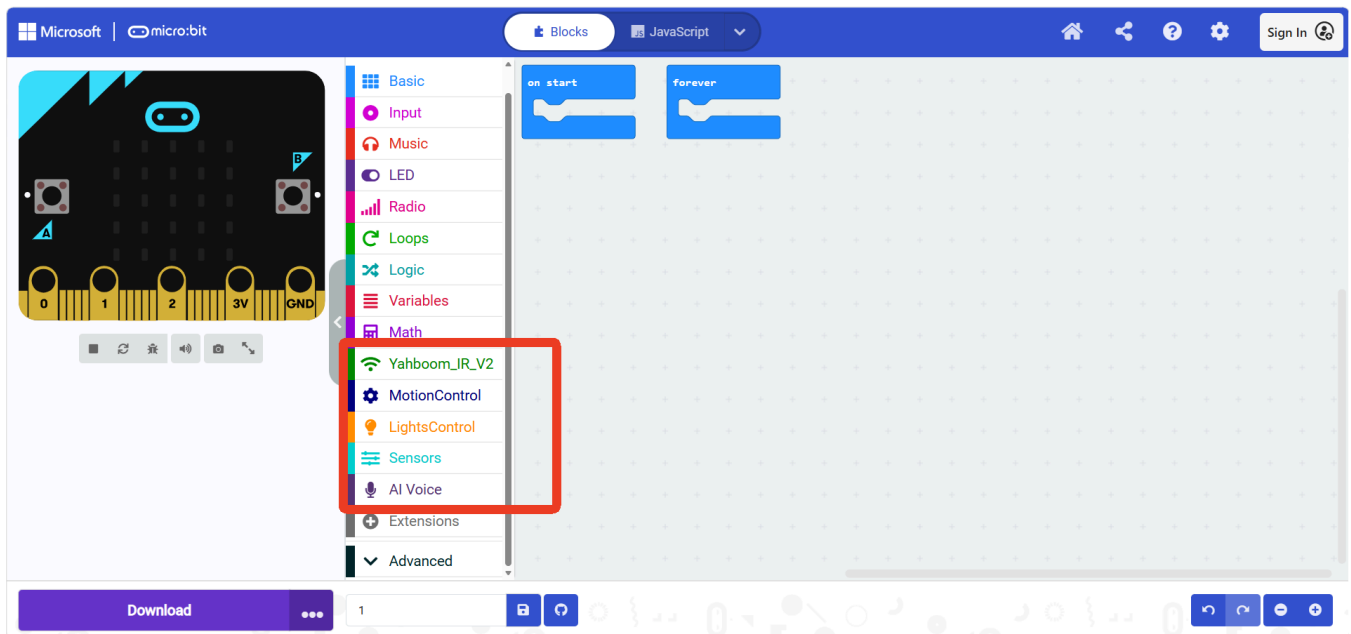
2. Enter <https://github.com/yahboomrdc/Lastbit-Lib> in the search box, then click "Search".



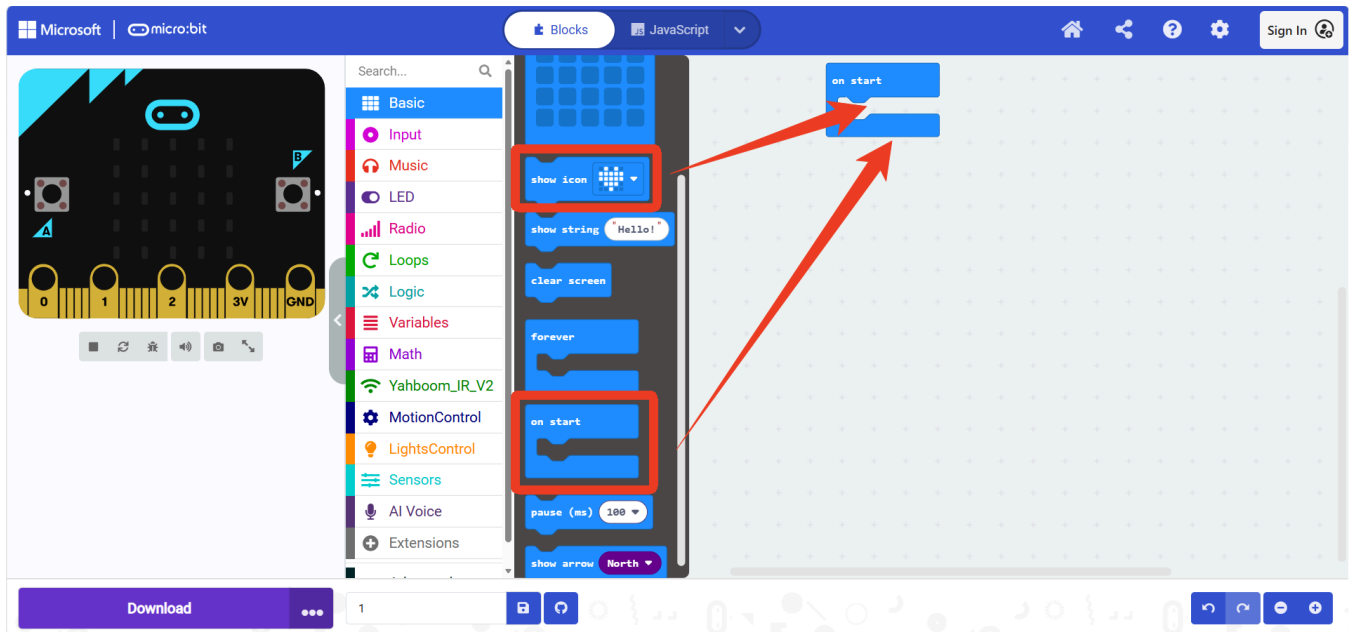
3. Select the Lastbit extension from the search results and import it.



4. After the import is complete, you can view the newly added categories in the category bar, indicating successful import.



5. Click the "Basic" category, find the corresponding block, and drag it to the programming area.



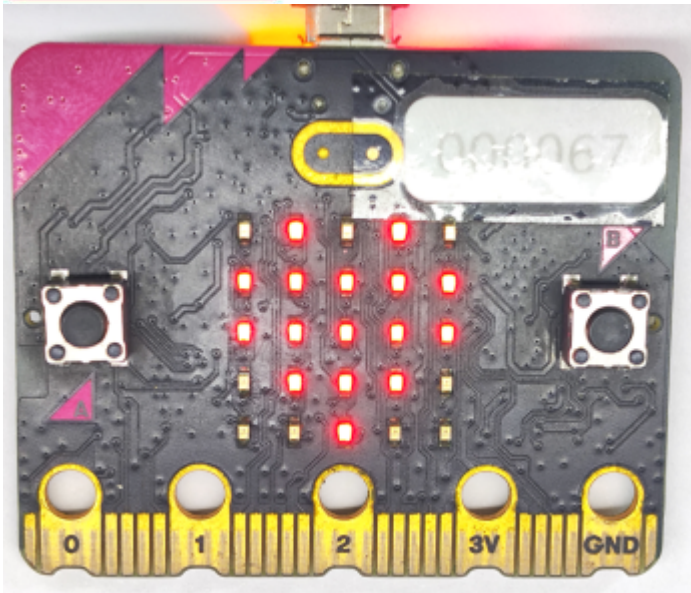
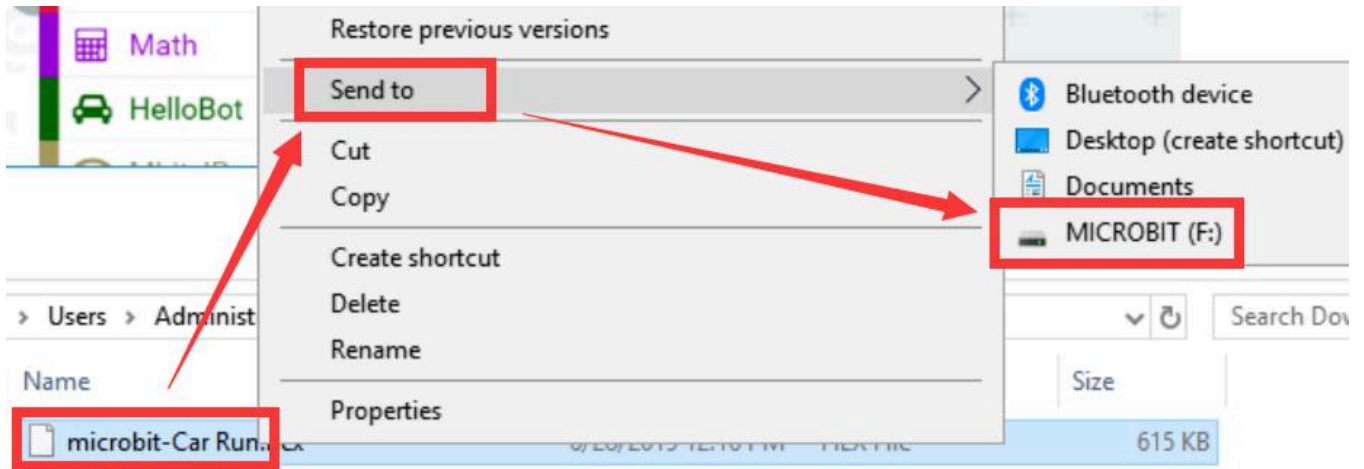
4. Flashing Programs

4.1 Flashing Example Programs

1. Connect Micro:bit to computer using a MicroUSB data cable. The computer will recognize the Micro:bit drive.

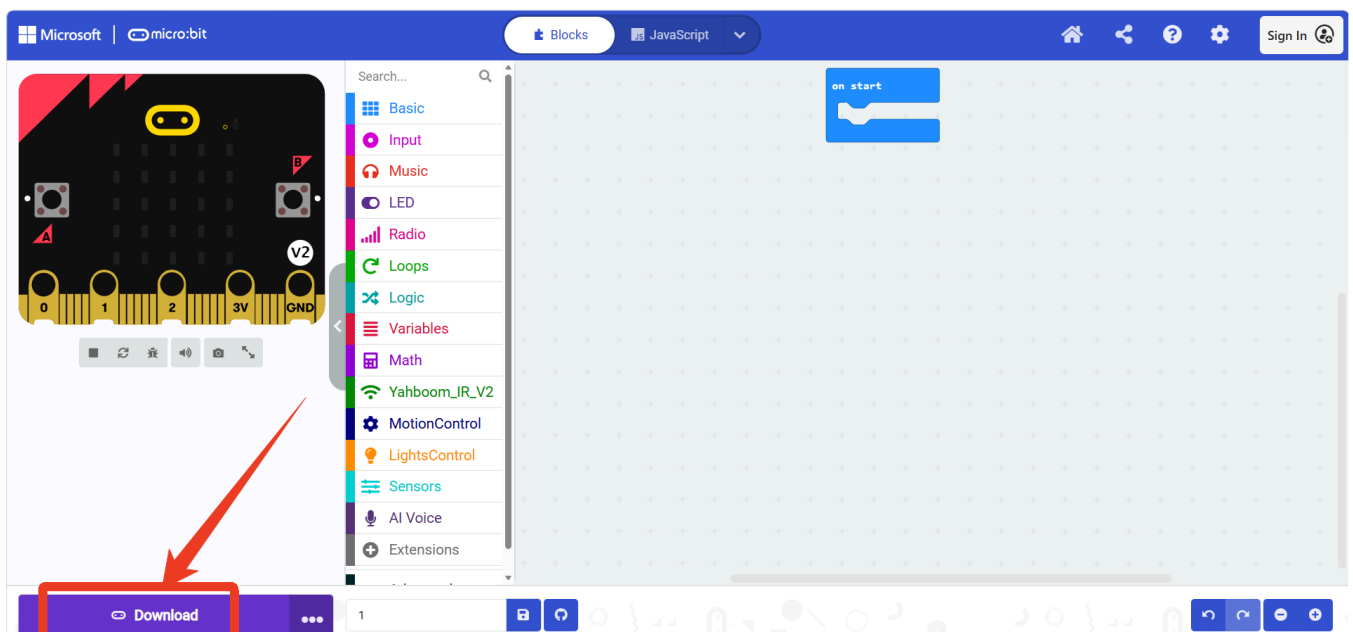


2. Copy the `.hex` program we provided to the Micro:bit drive.
3. During flashing, the orange indicator light on the back of the Micro:bit board will blink. After flashing is complete, the indicator light stops blinking, indicating the program has been successfully written.

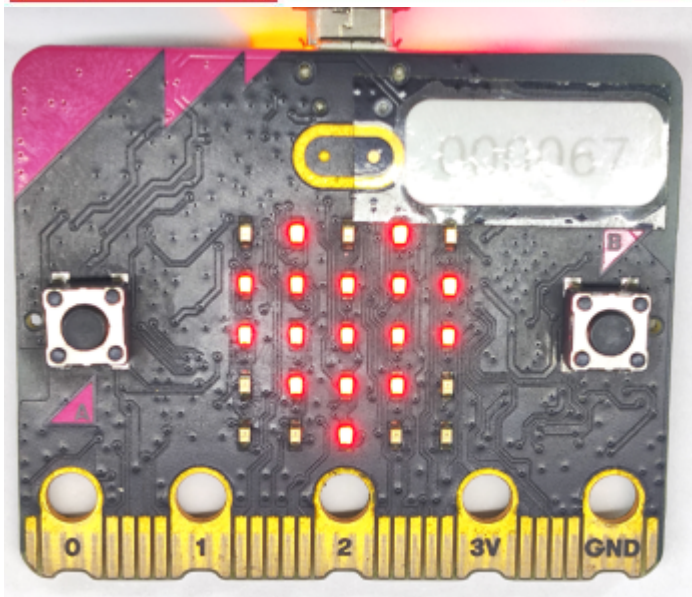
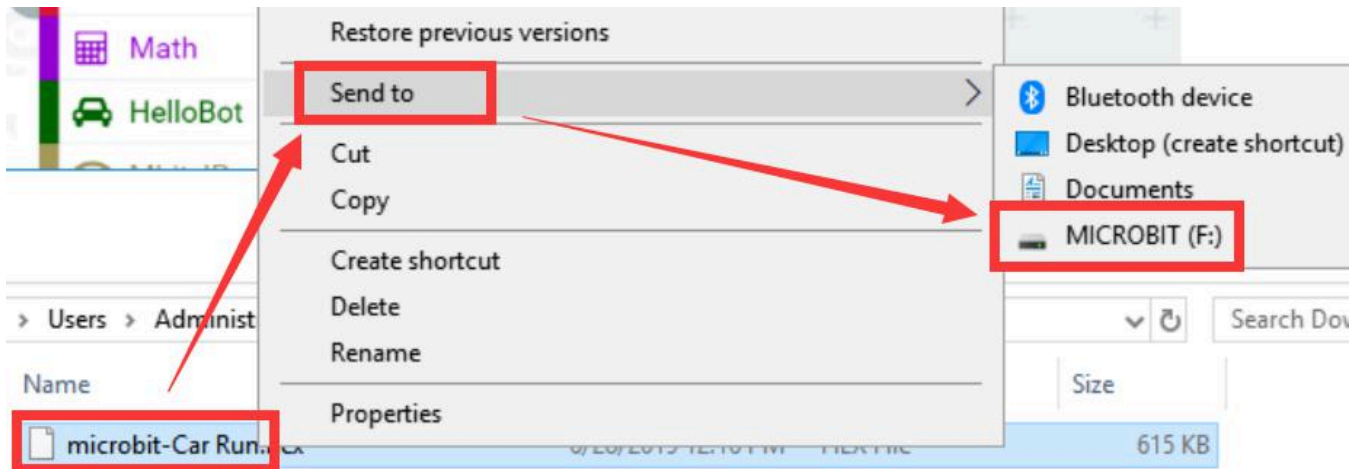


4.2 Flashing Custom Programs

1. Refer to the "Importing Lastbit Exclusive Extension Package" method above to import the extension package and complete program writing.
2. Click the "Download" or "Save" button to download the program to computer or save directly to the Micro:bit drive.



3. After generating the `.hex` file, copy it to the Micro:bit drive to complete flashing.
4. During flashing, the orange indicator light on the back of the Micro:bit board will blink. After flashing is complete, the indicator light stops blinking, indicating the program has been successfully written.



5. Experimental Phenomena

After flashing is complete, you can run the program on Micro:bit and observe the corresponding execution effects. If using a simulator, you can preview the running results on the page first.

6. Precautions

- The flashing interface is located on the Micro:bit board, not the Charging port on the expansion board.
- It is recommended to turn off the car expansion board power switch before flashing, and disconnect the data cable after flashing is complete.
- AI voice module extension examples require optional voice module selection to use.